DSE-3: Data Science – (Unit-2)

R Programming Basics: "R is an interpreted computer programming language which was developed by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand." It is a software environment used to analyze **statistical information**, **graphical representation**, **reporting**, and **data modelling**.

R not only allows us to do branching and looping but also allows to do modular programming using functions. R allows integration with the procedures written in the C, C++, .Net, Python, and FORTRAN languages to improve efficiency.

In the present era, R is one of the most important tool which is used by researchers, data analyst, statisticians, and marketers for retrieving, cleaning, analyzing, visualizing, and presenting data.

Applications of R: There are several-applications available in real-time. Some of the area where R is used are:

- Facebook
- Google
- Twitter
- Flipkart
- Amazon etc

Installation of R in Windows: Following are the steps used to install R in Windows:

Step 1: First, we have to download the R setup from **https://cran.r-project.org/bin/windows/base/old/3.4.1/**

Step 2: When we click on **Download R for windows**, our downloading will be started of R setup. Once the downloading is finished, we have to run the setup of R in the following way:

i) Select the path where we want to download the R and proceed to Next.
ii) Select all components which we want to install, and then we will proceed to **Next**.
iii) In the next step, we have to select either customized startup or accept the default, and then we proceed to **Next**.
iv) When we proceed to next, our installation of R in our system will get started:
v) In the last, we will click on finish to successfully install R in our system.

Installation of R Studio in Windows:

Step 1: Visit the RStudio official site **https://www.rstudio.com/** and click on **Download RStudio**.

Step 2: Select the RStudio desktop for open-source and click on download.

Step 3: Select the appropriate installer. When we select the installer, our downloading of RStudion setup will start.

Step 4: Run the setup in the following way:

i) Click on Next.
ii) Click on Install.
iii) Click on finish.
iv) RStudio is ready to work.

Advantages of R:

1. **Open Source :** An open-source language is a language on which we can work without any need for a license or a fee. R is an open-source language. We can contribute to the development of R by optimizing our packages, developing new ones, and resolving issues.
2. **Platform Independent:** R is a platform-independent language or cross-platform programming language which means its code can run on all operating systems. R enables programmers to develop software for several competing platforms by writing a program only once. R can run quite easily on Windows, Linux, and Mac.
3. **Machine Learning Operations:** R allows us to do various machine learning operations such as classification and regression. For this purpose, R provides various packages and features for developing the artificial neural network. R is used by the best data scientists in the world.
4. **Exemplary support for data wrangling:** R allows us to perform data wrangling. R provides packages such as dplyr, readr which are capable of transforming messy data into a structured form.
5. **Quality plotting and graphing:** R simplifies quality plotting and graphing. R libraries such as ggplot2 and plotly advocates for visually appealing and aesthetic graphs which set R apart from other programming languages.
6. **The array of packages:** R has a rich set of packages. R has over 10,000 packages in the CRAN repository which are constantly growing. R provides packages for data science and machine learning operations.
7. **Statistics:** R is mainly known as the language of statistics. It is the main reason why R is predominant than other programming languages for the development of statistical tools.
8. **Continuously Growing:** R is a constantly evolving programming language. Constantly evolving means when something evolves, it changes or develops over time, like our taste in music and clothes, which evolve as we get older. R is a state of the art which provides updates whenever any new feature is added.

Disadvantages of R:

1. **Data Handling:** In R, objects are stored in physical memory. It is in contrast with other programming languages like Python. R utilizes more memory as compared to Python. It requires the entire data in one single place which is in the memory. It is not an ideal option when we deal with Big Data.
2. **Basic Security:** R lacks basic security. It is an essential part of most programming languages such as Python. Because of this, there are many restrictions with R as it cannot be embedded in a web-application.
3. **Complicated Language:** R is a very complicated language, and it has a steep learning curve. The people who don't have prior knowledge or programming experience may find it difficult to learn R.
4. **Weak Origin:** The main disadvantage of R is, it does not have support for dynamic or 3D graphics. The reason behind this is its origin. It shares its origin with a much older programming language "S."
5. **Lesser Speed:** R programming language is much slower than other programming languages such as MATLAB and Python. In comparison to other programming language, R packages are much slower.

Syntax of R Programming: R Programming is a very popular programming language which is broadly used in data analysis. The way in which we define its code is quite simple. The "Hello World!" is the basic program for all the languages, and now we will understand the syntax of R programming with "Hello world" program. We can write our code either in command prompt, or we can use an R script file.

R Command Prompt: It is required that we have already installed the R environment set up in our system to work on the R command prompt. After the installation of R environment setup, we can easily start R command prompt by typing R in our Windows command prompt. When we press enter after typing R, it will launch interpreter, and we will get a prompt on which we can code our program.

"Hello, World!" Program

The code of "Hello World!" in R programming can be written as:

```
>string <-"Hello World!"
>print(string)
```

Output:

```
[1] "Hello World!"
```

In the above code, the first statement defines a **string variable** string, where we assign a string "Hello World!". The next statement print() is used to print the value which is stored in the variable string.

R Script File: The R script file is another way on which we can write our programs, and then we execute those scripts at our command prompt with the help of R interpreter known as **Rscript**. We make a text file and write the following code. We will save this file with **.R** extension as:

Demo.R

```
string <-"Hello World!"
print(string)
```

To execute this file in Windows and other operating systems, write **Rscript Demo.R** in the command prompt. When we press enter it will give us the following output:

```
[1] "Hello World!"
```

Comments: In R programming, comments are the programmer readable explanation in the source code of an R program. The purpose of adding these comments is to make the source code easier to understand. These comments are generally ignored by compilers and interpreters.

In R programming there is only single-line comment. R doesn't support multi-line comment. But if we want to perform multi-line comments, then we can add our code in a false block.

Single-line comment

```
#My First program in R programming
string <-"Hello World!"
print(string)
```

Multi-line comment

```
#Trick for multi-line comment
if(FALSE) {
  "R is an interpreted computer programming language which was created by
   Ross Ihaka and Robert Gentleman at the University of Auckland, New Zeala
   nd"
}
#My First program in R programming
string <-"Hello World!"
print(string)
```

Data Types in R Programming: Variables are the reserved memory location to store values. As we create a variable in our program, some space is reserved in memory.

In R, there are several data types such as integer, string, etc. The operating system allocates memory based on the data type of the variable and decides what can be stored in the reserved memory.

Following are some of the data types used in R.

Data type	Example	Description
Logical	True, False	It is a special data type for data with only two possible values which can be construed as true/false.
Numeric	12,32,112,5432	Decimal value is called numeric in R, and it is the default computational data type.
Integer	3L, 66L, 2346L	Here, L tells R to store the value as an integer,
Complex	Z=1+2i, t=7+3i	A complex value in R is defined as the pure imaginary value i.
Character	'a', "good", "TRUE", '35.4'	In R programming, a character is used to represent string values. We convert objects into character values with the help ofas.character() function.
Raw		A raw data type is used to holds raw bytes.

Example: Create a Rscript file Datatype.R to illustrate R Datatypes

```
#Logical Data type
variable_logical<- TRUE
cat(variable_logical,"\n")
cat("The data type of variable_logical is ",class(variable_logical),"\n\n")

#Numeric Data type
variable_numeric<- 3532
cat(variable_numeric,"\n")
cat("The data type of variable_numeric is ",class(variable_numeric),"\n\n")

#Integer Data type
variable_integer<- 133L
cat(variable_integer,"\n")
cat("The data type of variable_integer is ",class(variable_integer),"\n\n")

#Complex Data type
variable_complex<- 3+2i
cat(variable_complex,"\n")
cat("The data type of variable_complex is ",class(variable_complex),"\n\n")

#Character Data type
variable_char<- "Learning r programming"
cat(variable_char,"\n")
cat("The data type of variable_char is ",class(variable_char),"\n\n")

#Raw Data type
variable_raw<- charToRaw("ABCDEFGHIJKLMNOP  abcdefghijklmnop")
```

```
cat(variable_raw,"\n")
cat("The data type of variable_char is ",class(variable_raw),"\n\n")
```

When we execute the above program, it will give us the following output:

```
C:\Users\mkm>Rscript Datatatype.R
TRUE
The data type of variable_logical is logical

3532
The data type of variable_numeric is numeric

133
The data type of variable_integer is integer

3+2i
The data type of variable_complex is complex

Learning r programming
The data type of variable_char is character

41 42 43 44 45 46 47 48 49 4a 4b 4c 4d 4e 4f 50 20 20 61 62 63 64 65 66 67 68 69
6a 6b 6c 6d 6e 6f 70
The data type of variable_char is raw
```

Data types of variable: R programming is a dynamically typed language, which means that we can change the data type of the same variable again and again in our program. Because of its dynamic nature, a variable is not declared of any data type. It gets the data type from the R-object, which is to be assigned to the variable. We can check the data type of the variable with the help of the class() function.

Data Structures in R Programming: R has many data structures, which include:

1. Atomic Vector
2. List
3. Array
4. Matrices
5. Data Frame
6. Factors

Vectors: A vector is the basic data structure in R, or we can say vectors are the most basic R data objects. There are six types of atomic vectors such as logical, integer, character, double, and raw. **"A vector is a collection of elements which is most commonly of mode character, integer, logical or numeric"** A vector can be one of the following two types:

1. Atomic vector
2. Lists

List: In R, **the list** is the container. Unlike an atomic vector, the list is not restricted to be a single mode. A list contains a mixture of data types. The list is also known as generic vectors because the element of the list can be of any type of R object. **"A list is a special type of vector in which each element can be a different type."**

We can create a list with the help of **list() function**. We can use **vector() function** to create a required length empty list.

Arrays: There is another type of data objects which can store data in more than two dimensions known as arrays. **"An array is a collection of a similar data type with contiguous memory allocation."** Suppose, if we create an array of dimension (2, 3, 4) then it creates four rectangular matrices of two rows and three columns. In R, an array is created with the help of **array()** function. This function takes a vector as an input and uses the value in the dim parameter to create an array.

Matrices: A matrix is an R object in which the elements are arranged in a two-dimensional rectangular layout. In the matrix, elements of the same atomic types are contained. For mathematical calculation, this can use a matrix containing the numeric element. A matrix is created with the help of the **matrix() function** in R.

Data Frames: A **data frame** is a two-dimensional array-like structure, or we can say it is a table in which each column contains the value of one variable, and row contains the set of value from each column. Following are the characteristics of a data frame:

1. The column name will be non-empty.
2. The row names will be unique.
3. A data frame stored numeric, factor or character type data.
4. Each column will contain same number of data items.

Factors: **Factors** are also data objects that are used to categorize the data and store it as levels. Factors can store both strings and integers. Columns have a limited number of unique values so that factors are very useful in columns. It is very useful in data analysis for statistical modelling. Factors are created with the help of **factor() function** by taking a vector as an input parameter.

Variables in R Programming:

Variables are used to store the information to be manipulated and referenced in the R program. The R variable can store an atomic vector, a group of atomic vectors, or a combination of many R objects.

Language like C++ is statically typed, but R is a dynamically typed, means it check the type of data type when the statement is run.

Assignment of variable: In R programming, there are three operators which we can use to assign the values to the variable. We can use **leftward**, **rightward**, and **equal_to operator** for this purpose.

There are two functions which are used to print the value of the variable i.e., **print()** and **cat().** The cat() function combines multiples values into a continuous print output.

```
# Assignment using equal operator.
variable.1 = 124

# Assignment using leftward operator.
```

```
variable.2 <- "Learn R Programming"

# Assignment using rightward operator.
133L -> variable.3

print(variable.1)
cat ("variable.1 is ", variable.1 ,"\n")
cat ("variable.2 is ", variable.2 ,"\n")
cat ("variable.3 is ", variable.3 ,"\n")
```

When we execute the above code in our R command prompt, it will give us the following output:

```
 [1] 124
variable.1 is  124
variable.2 is  Learn R Programming
variable.3 is  133
```

Control structures in R:

R if Statement: The syntax of if statement in R is as follows:

```
if(boolean_expression) {
      // If the Boolean expression is true, then statement(s) will be executed.
}
```

Example-1

```
x <- 20
24 -> y
count=0
if(x<y) {
   cat(x,"is a smaller number\n")
   count=1
}
if(count==1){
   cat("Block is successfully executed")
}
```

Output:

```
20 is a smaller number
Block is successfully executed
```

Example-2

```
x = 24L
y = "shubham"
if(is.integer(x))  {
   cat(x,"is an Integer\n")
}
```

Output:

```
24 is an Integer
```

If-else statement: The basic syntax of If-else statement is as follows:

```
if(boolean_expression) {
  // statement(s) will be executed if the boolean expression is true.
} else {
  // statement(s) will be executed if the boolean expression is false.
}
```

Example-3

```
x <-c("Hardwork","is","the","key","of","success")
if("key" %in% x) {
        print("key is found")
} else {
        print("key is not found")
}
```

Output:

```
[1] "key is found"
```

R Switch Statement: A switch statement is a selection control mechanism that allows the value of an expression to change the control flow of program execution via map and search.

The switch statement is used in place of long if statements which compare a variable with several integral values. It is a multi-way branch statement which provides an easy way to dispatch execution for different parts of code. This code is based on the value of the expression.

This statement allows a variable to be tested for equality against a list of values. A switch statement is a little bit complicated. To understand it, we have some key points which are as follows:

- If expression type is a character string, the string is matched to the listed cases.
- If there is more than one match, the first match element is used.
- No default case is available.
- If no case is matched, an unnamed case is used.

There are basically two ways in which one of the cases is selected:

1) Based on Index: If the cases are values like a character vector, and the expression is evaluated to a number than the expression's result is used as an index to select the case.

2) Based on Matching Value: When the cases have both case value and output value like ["case_1"="value1"], then the expression value is matched against case values. If there is a match with the case, the corresponding value is the output.

The basic syntax of switch statement is as follows:

```
switch(expression, case1, case2, case3....)
```

Example 1

```
x <-switch(3,  "1st",  "2nd",  "3rd",   "4th" )
print(x)
```

Output:

```
[1] "3rd"
```

Example 2

```
y = "18"
x = switch(y, "9"="Ankit", "12"="Rupa", "18"="Nisha", "21"="Jiban")
print (x)
```

Output:

```
[1] "Nisha"
```

For loop in R Programming: For loop is a way to repeat a sequence of instructions under certain conditions. It allows us to automate parts of our code which need repetition. In simple words, a for loop is a repetition control structure. It allows us to efficiently write the loop that needs to execute a certain number of times.

In R, a for loop is defined as follows:

1. It starts with the keyword **for** like C or C++.
2. Instead of initializing and declaring a loop counter variable, we declare a variable which is of the same type as the base type of the vector, matrix, etc., followed by a colon, which is then followed by the array or matrix name.
3. In the loop body, use the loop variable rather than using the indexed array element.

syntax of for loop in R:

```
for (value in vector) {
  statements
}
```

Example 1: Iterate all the elements of a vector and print the current value.

```
# Create fruit vector
fruit <- c('Apple', 'Orange',"Guava", 'Pinapple', 'Banana','Grapes')
# Create the for statement
for ( i in fruit){
   print(i)
}
```

Output:

```
[1] "Apple"
[1] "Orange"
[1] "Guava"
[1] "Pinapple"
[1] "Banana"
[1] "Grapes"
```

Example 2: Create a non-linear function with the help of the polynomial of x between 1 and 5 and store it in a list.

```
# Creating an empty list
list <- c()
# Creating a for statement to populate the list
for (i in seq(1, 5, by=1)) {
  list[[i]] <- i*i
}
print(list)
```

Output: [1] 1 4 9 16 25

Example 3: For loop over a matrix

```
# Creating a matrix
mat <- matrix(data = seq(10, 15, by=1), nrow = 3, ncol =2,
                                                  byrow = TRUE)
# Creating the loop with r and c to iterate over the matrix
for (r in 1:nrow(mat))
   for (c in 1:ncol(mat))
       print(paste("mat[", r, "," ,c, "]=", mat[r,c]))
print(mat)
```

Output:

```
[1] "mat[ 1 , 1 ]= 10"
[1] "mat[ 1 , 2 ]= 11"
[1] "mat[ 2 , 1 ]= 12"
[1] "mat[ 2 , 2 ]= 13"
[1] "mat[ 3 , 1 ]= 14"
[1] "mat[ 3 , 2 ]= 15"
     [,1] [,2]
[1,]   10   11
[2,]   12   13
[3,]   14   15
```

Example 4: For loop over a list

```
# Create a list with three vectors
fruit <- list(Basket = c('Apple', 'Orange',"Guava", 'Pinapple', 'Banana','Grapes'),
Money = c(10, 12, 15), purchase = TRUE)
for (p  in fruit)
   print(p)
```

Output:

```
[1] "Apple"    "Orange"   "Guava"    "Pinapple" "Banana"   "Grapes"
[1] 10 12 15
[1] TRUE
```

Example 5: count the number of even numbers in a vector.

```
x <- c(2,5,3,9,8,11,6,44,43,47,67,95,33,65,12,45,12)
count <- 0
for (val in x)
      if(val %% 2 == 0)  count = count+1
print(count)
```

Output:

```
[1] 6
```

R repeat loop: A repeat loop is used to iterate a block of code. **It is a special type of loop in which there is no condition to exit from the loop.**
For exiting, we include a break statement with a user-defined condition. This property of the loop makes it different from the other loops. A repeat loop constructs with the help of the repeat keyword in R.

Syntax of the repeat loop:

```
repeat {
        commands
        if(condition) {
            break
        }
}
```

Example 1:

```
v <- c("Hello","repeat","loop")
cnt <- 1
repeat {
        print(v)
        cnt <- cnt+1
        if(cnt > 3)   break
}
```

Output:

```
[1] "Hello"  "repeat" "loop"
[1] "Hello"  "repeat" "loop"
[1] "Hello"  "repeat" "loop"
```

R while loop: A while loop is a type of control flow statements which is used to iterate a block of code several numbers of times. The “while loop” terminates when the value of the Boolean expression becomes false. In while loop, firstly the condition will be checked and then after the body of the statement will execute. In this statement, the condition will be checked n+1 time, rather than n times.

The basic syntax of while loop is as follows:

```
while (test_expression) {
  statement
}
```

Example 1:

```
v <- c("while loop","example")
cnt <- 2
while (cnt < 7) {
   print(v)
   cnt = cnt + 1
}
```

Output:

```
[1] "while loop" "example"
[1] "while loop" "example"
[1] "while loop" "example"
[1] "while loop" "example"
[1] "while loop" "example"
```

R Functions: A set of statements which are organized together to perform a specific task is known as a function. R provides a series of in-built functions, and it allows the user to create their own functions. Functions are used to perform tasks in the modular approach. Functions are used to avoid repeating the same task and to reduce complexity. To understand and maintain our code, we logically break it into smaller parts using the function.

An R function is created by using the keyword function. The following is syntax of R function:

```
func_name <- function(arg_1, arg_2, ...) {
   Function body
}
```

Components of Functions: There are four components of function, which are as follows:

Function Name: The function name is the actual name of the function. In R, the function is stored as an object with its name.

Arguments: In R, an argument is a placeholder. In function, arguments are optional which means a function may or may not contain arguments, and these arguments can have default values also. We pass a value to the argument when a function is invoked.

Function Body: The function body contains a set of statements which defines what the function does.

Return value: It is the last expression in the function body which is to be evaluated.

Function Types: Similar to the other languages, R also has two types of function, i.e. **Built-in Function** and **User-defined Function**. In R, there are lots of built-in functions which we can directly call in the program without defining them. R also allows us to create our own functions.

Built-in function: The functions which are already created or defined in the programming framework are known as built-in functions. User doesn't need to create these types of functions, and these functions are built into an application. End-users can access these functions by simply calling it. R have different types of built-in functions such as seq(), mean(), max(), and sum(x) etc.

Creating sequence of numbers from 32 to 46.

```
print(seq(32,46))
```

Output: 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46

Finding the mean of numbers from 22 to 80.

```
print(mean(22:80))
```

Output: 51

Finding the sum of numbers from 41 to 70.

```
print(sum(41:70))
```

Output: 1665

User-defined function:	**Example:**	**Output:**

R allows us to create our own function in our program. A user defines a user-defined function to fulfil the requirement of user. Once these functions are created, we can use these functions like in-built functions.	**# Creating a function without an argument.**
	f1 <- function() {
	  for(i in 1:5) {
	  print(i^2)
	  }
	}
	f1()
		[1] 1
		[1] 4
		[1] 9
		[1] 16
		[1] 25

Function calling with an arguments: We can easily call a function by passing an appropriate argument in the function. Let see an example to see how a function is called.

# Creating a function to print squares of numbers from 1 to n	**Output:**

f2 <- function(n) {
  for(i in 1:n) {
  x <- i^2
  print(x)
  }}
f2(5)
	[1] 1
	[1] 4
	[1] 9
	[1] 16
	[1] 25

Function calling with Argument Values: We can supply the arguments to a function call in the same sequence as defined in the function or can supply in a different sequence but assigned them to the names of the arguments.

# Creating a function with arguments	**Output:**

f3 <- function(x,y,z) {
  result <- x * y + z
  print(result)
}
f3(2,3,8)
# Calling the function by names of the arguments. 	
f3(z = 2, x = 5, y = 3)
	[1] 14
	[1] 17

Function calling with default arguments: To get the default result, we assign the value to the arguments in the function definition, and then we call the function without supplying argument. If we pass any argument in the function call, then it replaces the default value of the argument in the function definition.

Example:

```
# Creating a function with arguments
f4 <- function(x = 5, y = 7) {
   result <- x * y
   print(result)
}
f4()
f4(4,6)
```

Output:
```
[1] 35
[1] 24
```

R Built-in Functions: The functions which are already created or defined in the programming framework are known as a built-in function. R has a rich set of functions that can be used to perform almost every task for the user. These built-in functions are divided into the following categories based on their functionality.

Math Functions: R provides the various mathematical functions to perform the mathematical calculation. These mathematical functions are very helpful to find absolute value, square value and much more calculations. Some of the mathematical functions in R are explained below:

Function	Description	Example
abs(x)	It returns the absolute value of input x.	x<- -4 print(abs(x)) **Output:** [1] 4
sqrt(x)	It returns the square root of input x.	x<- 4 print(sqrt(x)) **Output:** [1] 2
ceiling(x)	It returns the smallest integer which is larger than or equal to x.	x<- 4.5 print(ceiling(x)) **Output:** [1] 5
floor(x)	It returns the largest integer, which is smaller than or equal to x.	x<- 2.5 print(floor(x)) **Output:** [1] 2
trunc(x)	It returns the truncate value of input x.	x<- c(1.2, 2.5, 8.1) print(trunc(x)) **Output:** [1] 1 2 8
round(x, n)	It returns round value of input x up to n decimal places.	x<- 4.346 print(round(x,2)) **Output:** 4.35
log(x)	It returns natural logarithm of input x.	x<- 4

		print(log(x)) **Output:** [1] 1.386294
log10(x)	It returns common logarithm of input x.	x<- 10 print(log10(x)) **Output:** [1] 1
exp(x)	It returns exponent.	x<- 4 print(exp(x)) **Output:** [1] 54.59815

String Function: R provides various string functions to perform string manipulation tasks. These string functions allow us to extract sub string from string, search pattern etc. Some of the strings handling functions in R are explained below:

Function	**Description**	**Example**
substr(x, start=n1,stop=n2)	It is used to extract substrings in a character vector.	a <- "987654321" substr(a, 3, 6) **Output:** [1] "7654"
grep(pattern, x , ignore.case=FALSE, fixed=FALSE)	It searches for pattern in x.	st1<- c('abcd','bdcd','abcdabcd') pattern<- 'abc' print(grep(pattern, st1)) **Output:** [1] 1 3
sub(pattern, replacement, x, ignore.case =FALSE, fixed=FALSE)	It finds pattern in x and replaces it with replacement (new) text.	st1<- "England is beautiful" sub("England", "UK", st1) **Output:** [1] "UK is beautiful"
paste(..., sep="")	It concatenates strings after using sep string to separate them.	paste('one',2,'three',4,'five') **Output:** [1] "one 2 three 4 five"
strsplit(x, split)	It splits the elements of character vector x at split point.	a<-"Split all the words" print(strsplit(a, " ")) **Output:** [[1]] [1] "Split" "all" "the" "words"
tolower(x)	It is used to convert the string into lower case.	st1<- "shuBHAm" print(tolower(st1)) **Output:** [1] "shubham"
toupper(x)	It is used to convert the string into upper case.	st1<- "shuBHAm" print(toupper(st1)) **Output:** [1] "SHUBHAM"

Statistical Functions: R provides various statistical functions to perform statistical task. Some of the statistical functions in R are explained below:

Function	Description	Example
mean(x, trim=0, na.rm=FALSE)	It is used to find the mean for x object	a<-c(0:10, 40) xm<-mean(a) print(xm) **Output:** [1] 7.916667
sd(x)	It returns standard deviation of an object.	a<-c(0:10, 40) xm<-sd(a) print(xm) **Output:** [1] 10.58694
median(x)	It returns median.	a<-c(0:10, 40) xm<-median(a) print(xm) **Output:** [1] 5.5
range(x)	It returns range.	a<-c(0:10, 40) xm<-range(a) print(xm) **Output:** [1] 0 40
sum(x)	It returns sum.	a<-c(0:10, 40) xm<-sum(a) print(xm) **Output:** [1] 95
diff(x, lag=1)	It returns differences with lag indicating which lag to use.	a<-c(0:10, 40) xm<-diff(a) print(xm) **Output:** [1] 1 1 1 1 1 1 1 1 1 1 30
min(x)	It returns minimum value.	a<-c(0:10, 40) xm<-min(a) print(xm) **Output:** [1] 0
max(x)	It returns maximum value	a<-c(0:10, 40) xm<-max(a) print(xm) **Output:** [1] 40

Scoping rules: The scope of a variable is nothing more than the place in the code where it is referenced and visible. The mostly used scoping rule in R is ***lexical scoping or Static Scoping***.

Lexical Scoping: Lexical scoping in R means the ways in which the values of free variables are searched for in the environment in which the function was defined. Searching for the value of the free variable uses the following rules:

- If the value of the symbol is not found in the environment in which a function was defined, then the search is continued in the parent environment.
- The search continues the sequence of the parent environment until the users hit the top-level environment; this usually the global environment (workspace) or the namespace of a package.
- If the value for a given symbol cannot be found once the **empty environment (i.e. the environment without any parent environment)** has arrived, then an error is thrown.

Example:

```
f <- function(x, y)
{
   x * y * z
}
```

Here,
x and y are formal arguments
z is the free variable
Therefore, the scoping rules of the language determine how values are assigned to free variables.

Free variables are not formal arguments and not local variables that are assigned inside of the function body.

Principles of Lexical Scoping: There are mainly three basic principles of R's implementation of lexical scoping:

1. Name Masking
2. Functions vs variables
3. A fresh start

Name Masking: If a variable or function is not defined inside the function, It looks one level up. If it is still not found then it searches the next higher level and finally in the global environment and then to the other loaded packages.

Example:

```
c <- 10
f <- function(a, b) {  a + b + c }
f(8, 5)
```

Output:

```
[1] 23
```

Example:

```
a <- 10
b <- function() {   c <- 11
                    c(a, c)    }
b()
```

Output:

```
[1] 10 11
```

The same rules apply if a function is defined inside another function.

Example:

```
a <- 10
g <- function(){     b <- 20
                     h <- function(){    c <- 30
                                         c(a, b, c)    }
                     h()  }
g()
```

Output:

```
[1] 10 20 30
```

Functions vs Variables: *The same principles apply regardless of the type of associated value — finding functions works exactly the same way as finding variables:*

Example:

```
a <- function(x) { x * 10 }
b <- function(){
     a <- function(x) {x + 10}
     a(12)       }
b()
a(12)
```

Output:

```
[1] 22
[1] 120
```

A Fresh Start: When a function is called, a new environment is created every time. Each acknowledgement is completely independent because a function cannot tell what has happened when it was run last time.

Example:

```
z = 10
a <- function(){
      if(!exists("z")){   z <- 10   }
      else{   z <- z+10  }
  z               }
a()
```

Output:

```
[1] 20
```